

Part 1: Theoretical Questions

Submit the solution to this part as part1.pdf. We can't stress this enough: the file *has to be a PDF file*.

Question 1.1:

1. **Explain the following programming paradigms:**

- (a) **Imperative [5 points]:** The control flow is an explicit sequence of commands.
- (b) **Procedural [5 points]:** Just like Imperative but organized around hierarchies of nested procedure calls.
- (c) **Functional [5 points]:** computation is performed by nesting function calls and composing functions together, all while avoiding mutations to any global state, and through the definition of function composition.

2. **How does the procedural paradigm improve over the imperative paradigm? [5 points]**

By introducing layers of abstraction via procedures, which interact through well-defined contracts and can encapsulate local variables.

3. **How does the functional paradigm improve over the procedural paradigm? [5 points]**

By discouraging the use of shared state and mutation, which simplifies debugging, testing, and parallel execution.

Question 1.2:

Consider the following TypeScript function, which processes an array of sales transactions to calculate the total revenue for each product category. The function follows these rules:

Apply a 10% discount to transactions where the quantity is greater than 5.

Exclude transactions with a total value (after discount) of less than 50.

Sum the total revenue for each category from the remaining transactions.

```
type Transaction = { category:
  string; price: number;
  quantity: number;
}

function calculateRevenueByCategory(transactions: Transaction[]): Record<string, number> { const
  revenueByCategory: Record<string, number> = {};

  for (const transaction of transactions) { // Apply 10% discount if quantity
    > 5 let total = transaction.price * transaction.quantity; if
    (transaction.quantity > 5) { total *= 0.9; // 10% discount
    }

    // Filter out transactions with total < 50 if (total >= 50) {
      if (!revenueByCategory[transaction.category]) { revenueByCategory[transaction.category] = 0;
      } revenueByCategory[transaction.category] += total; } }

  return revenueByCategory;
}
```

This function uses an imperative approach with loops and conditional statements.

Refactor the function `calculateRevenueByCategory` to adhere to the Functional Programming paradigm. Utilize the built-in array methods `map`, `filter`, and `reduce` to achieve the same functionality without explicit iteration and conditional checks. Rename it to `calculateRevenueByCategoryFP`.

Important: the function should have the same signature.

Note: there're no tests for this question, and it will not be executed. The task here is to write the code in a functional way.

(answer next page)

```

type Transaction = {
  category: string;
  price: number;
  quantity: number;
};

```

```

type Transaction2 = {
  category: string;
  price: number;
  quantity: number;
  total: number
};

```

```

const map2 = (t: Transaction[]): Transaction2[] => (t.map(t1 => ({category: t1.category,
  price: t1.price,
  quantity: t1.quantity,
  total: (t1.quantity > 5 ? t1.price * t1.quantity * 0.9 : t1.price * t1.quantity) })))

```

```

const filter2 = (t: Transaction2[]): Transaction2[] => (t.filter(t1 => t1.total >= 50))

```

```

const sum = (t: Transaction2[]): Record<string, number> =>
  t.reduce<Record<string, number>>((acc, { category, total }) => {
    acc[category] = (acc[category] || 0) + total;
    return acc;
  }, {});

```

```

const calculateRevenueByCategoryFP2 = (transactions: Transaction[]): Record<string, number> =>
  (sum(filter2(map2(transactions))))

```

Question 1.3:

Write the most specific types for the following expressions. Guidelines:

Arrays must be homogeneous.

Arithmetic operations must be performed on numbers.

Use generics where possible.

Avoid using any.

1. **(x, y) => x.some(y) [3 points]:** $_ <T> (x: T[], y: (z: T) => \text{boolean}) => \text{boolean}$
2. **x => x.map(y => y * 2) [3 points]:** $_ (x: \text{number}[]) => \text{number}[]$
3. **(x, y) => x.filter(y) [3 points]:** $_ <T> (x: T[], y: (val: T) => \text{boolean}) => T[]$
4. **x => x.reduce((acc, cur) => acc + cur, 0) [3 points]:** $_ (x: \text{number}[]) => \text{number}$
5. **(x, y) => x ? y[0] : y[1] [3 points]:** $_ <T> (x: \text{boolean}, y: T[]) => T$
6. **(f,g) => x => f(g(x+1)) [3 points]:** $_ <T1, T2> (f: (b: T1) => T2, g: (a: \text{number}) => T1) => (x: \text{number}) => T2$